

ASSESSMENT

Section A — Concept Application

SCENARIO: You are designing the database for a food delivery application. It stores restaurant details, menu items linked to each restaurant, and customer orders. You must decide which columns need NOT NULL constraints, which serve as PRIMARY KEYS, and which link tables via FOREIGN KEYS.

Question: Justify your constraint choices for at least three columns across two tables. What data integrity problems would arise if these constraints were not applied?

ANS:

In a food delivery database, constraints are used to maintain data accuracy and consistency.

1. **Restaurant_ID – PRIMARY KEY:**

Restaurant_ID should be the **PRIMARY KEY** in the Restaurant table because each restaurant must have a unique identifier. If this constraint is not applied, duplicate or unidentified restaurant records could occur.

2. **Restaurant_Name – NOT NULL:**

Restaurant_Name should be **NOT NULL** because every restaurant must have a name. Without this constraint, restaurants could be stored without names, making them difficult to identify.

3. **Menu_Item_ID – PRIMARY KEY:**

Menu_Item_ID should be the **PRIMARY KEY** in the Menu table because every menu item needs a unique identifier. Without it,

duplicate menu items could be created and individual items could not be reliably referenced.

4. **Restaurant_ID in Menu – FOREIGN KEY:**

Restaurant_ID in the Menu table should be a **FOREIGN KEY** referencing Restaurant_ID in the Restaurant table. This ensures that every menu item belongs to an existing restaurant. Without it, menu items could be linked to restaurants that do not exist, creating invalid or orphan records.

5. **Order_ID – PRIMARY KEY:**

Order_ID should be the **PRIMARY KEY** in the Orders table so that every order can be uniquely identified. Without it, duplicate order records could cause incorrect billing, delivery, or order tracking.

Therefore, **PRIMARY KEY** constraints provide unique identification, **NOT NULL** prevents missing essential information, and **FOREIGN KEY** constraints maintain valid relationships between tables. These constraints prevent duplicate, incomplete, and inconsistent data.

SCENARIO : You are managing the database for a food delivery platform. The operations team asks you to update the status of all orders placed more than 7 days ago that are still marked 'Pending' to 'Cancelled', and to permanently delete menu items marked 'Unavailable' from restaurants that have closed.

Question: Explain why the WHERE clause is essential in each DML statement. What could go wrong with an UPDATE or DELETE executed without a proper WHERE condition?

ANS:

The **WHERE clause** is essential in DML statements such as UPDATE and DELETE because it specifies exactly which records should be affected.

For example:

UPDATE Orders

SET Status = 'Cancelled'

WHERE Status = 'Pending'

AND Order_Date < CURRENT_DATE - INTERVAL 7 DAY;

Here, the WHERE clause ensures that only orders that are **Pending** and were placed **more than 7 days ago** are cancelled.

For deleting unavailable menu items:

DELETE FROM Menu

WHERE Status = 'Unavailable'

AND Restaurant_ID IN (

SELECT Restaurant_ID

FROM Restaurant

WHERE Status = 'Closed')

);

The WHERE clause ensures that only **Unavailable menu items belonging to closed restaurants** are permanently deleted.

If an UPDATE is executed without a proper WHERE clause, **all records in the table may be updated**, causing incorrect order statuses. Similarly, a DELETE without a proper WHERE clause can **delete every record in the table**, resulting in permanent data loss.

Therefore, the WHERE clause is important for **data accuracy, safety, and preventing accidental changes or deletion of valid records**.

SCENARIO : You are building a revenue dashboard for a food delivery startup. The management wants total revenue, average order value, and number of orders per restaurant — but only for restaurants that have processed more than 20 orders in the current month.

Question: Which SQL clauses and aggregate functions would you use to produce this report? Explain why HAVING must be used here rather than WHERE to filter by aggregate results.

ANS:

SCENARIO : You are developing an order history page. The orders table has order_id, customer_id, restaurant_id, and total_amount. The restaurants table has restaurant_id and name. The page must show order ID, restaurant name, and amount for every order — including any that reference a restaurant no longer in the table.

Question: Compare INNER JOIN and LEFT JOIN for this query. Which would you use and why? Describe what each join type does to records that have no match in the other table.

ANS:

An **INNER JOIN** returns only the records that have a matching record in both tables. If an order refers to a restaurant that no longer exists in the restaurants table, that order will **not appear** in the result.

Example:

```
SELECT o.order_id, r.name, o.total_amount
FROM orders o
INNER JOIN restaurants r
ON o.restaurant_id = r.restaurant_id;
```

A **LEFT JOIN** returns **all records from the left table (orders)**, even if there is no matching restaurant. For an order whose restaurant is missing, the restaurant name will be shown as **NULL**.

Example:

```
SELECT o.order_id, r.name, o.total_amount
FROM orders o
LEFT JOIN restaurants r
ON o.restaurant_id = r.restaurant_id;
```

I would use a **LEFT JOIN** because the requirement is to show **every order**, including orders whose restaurant no longer exists.

Difference:

- **INNER JOIN:** Unmatched orders are excluded.
- **LEFT JOIN:** Unmatched orders are included, with NULL for the missing restaurant information.

Therefore, **LEFT JOIN is the appropriate choice** for the order history page.

SCENARIO : You are building the analytics layer for a food delivery platform. A senior developer proposes a VIEW called top_restaurants showing restaurant name, total orders, and average rating — hiding raw ID columns from the front-end team.

Question: Justify why a VIEW is preferred over direct table access here. What are two limitations the front-end developer should be aware of when working with this VIEW?

ANS:

A **VIEW** is preferred over direct table access because it provides a simplified and controlled way to access data. The top_restaurants VIEW can show only the required information such as **restaurant name, total orders, and average rating**, while hiding raw ID columns and the complexity of the underlying tables.

Advantages of using a VIEW:

- It improves **security** by hiding unnecessary columns and raw table details.
- It simplifies queries for the front-end developer.
- It provides a consistent format for accessing analytics data.

Two limitations:

1. **Limited updates:** A VIEW based on calculations such as COUNT() and AVG() may not be directly updatable. The front-end developer generally cannot use it like a normal table to modify the underlying data.
2. **Dependency on base tables:** If the underlying tables or columns used by the VIEW are changed or removed, the VIEW may stop working or return errors.

Therefore, a VIEW is useful because it **simplifies access, improves security, and hides database complexity**, but developers should be aware of its update limitations and dependency on the underlying tables.

SCENARIO : You are implementing the payment module for a food delivery app. When a customer places an order, the system must deduct the amount from the customer's wallet balance and record the payment simultaneously. During load testing, the wallet deduction completes but the payment record insert occasionally fails, leaving balances inconsistent.

Question: Which database mechanism should have prevented this inconsistency? Explain how COMMIT, ROLLBACK, and SAVEPOINT work together to ensure both operations succeed or neither takes effect.

ANS:

The database mechanism that should prevent this inconsistency is a TRANSACTION using COMMIT and ROLLBACK. These ensure that both the wallet deduction and payment record insertion are treated as a single unit of work.

For example:

```
START TRANSACTION;
```

```
UPDATE Wallet
```

```
SET Balance = Balance - 500
```

```
WHERE Customer_ID = 101;
```

```
SAVEPOINT wallet_updated;
```

```
INSERT INTO Payments (Customer_ID, Amount, Status)
```

```
VALUES (101, 500, 'Success');
```

```
COMMIT;
```

- COMMIT: Permanently saves all changes made during the transaction. It should be executed only when both the wallet deduction and payment insertion succeed.
- ROLLBACK: Cancels all changes made during the transaction. If the payment insertion fails, ROLLBACK can undo the wallet deduction, so the balance remains correct.
- SAVEPOINT: Creates a temporary point inside a transaction. If an error occurs, the database can use ROLLBACK TO SAVEPOINT to undo changes made after that point without cancelling the entire transaction.

Therefore, if the payment record insertion fails, the transaction should ROLLBACK the wallet deduction. If both operations succeed, COMMIT saves them permanently. This follows the all-or-nothing principle of transactions and prevents inconsistent wallet and payment data.

Section B — Practical Coding Tasks

Task 1: Design and Populate a Restaurant Database

ANS:

```
CREATE DATABASE IF NOT EXISTS foodapp_db;
```

```
USE foodapp_db;
```

```
DROP TABLE IF EXISTS restaurants;
```

```
CREATE TABLE restaurants (  
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    city VARCHAR(50),  
    cuisine_type VARCHAR(50),  
    rating DECIMAL(3,1)  
);
```

```
INSERT INTO restaurants (name, city, cuisine_type, rating)  
VALUES  
( 'Spice Garden', 'Surat', 'Indian', 4.5),  
( 'Royal Dine', 'Mumbai', 'Chinese', 4.2),  
( 'Food Palace', 'Navsari', 'Gujarati', 4.7),
```

```
('Urban Tadka', 'Surat', 'Punjabi', 4.1),  
( 'Taste Hub', 'Ahmedabad', 'Italian', 4.6),  
( 'Green Leaf', 'Mumbai', 'South Indian', 4.0);
```

UPDATE restaurants

SET rating = 4.8

WHERE restaurant_id = 1;

DELETE FROM restaurants

WHERE restaurant_id = 6;

SELECT *

FROM restaurants

ORDER BY rating DESC;

```
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0048 seconds.)  
  
CREATE DATABASE IF NOT EXISTS foodapp_db;  
[ Edit inline ] [ Edit ] [ Create PHP code ]  
  
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)  
  
USE foodapp_db;  
[ Edit inline ] [ Edit ] [ Create PHP code ]  
  
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0025 seconds.)  
  
DROP TABLE IF EXISTS restaurants;  
[ Edit inline ] [ Edit ] [ Create PHP code ]  
  
⚠ Note: #1051 Unknown table 'foodapp_db.restaurants'  
  
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0078 seconds.)  
  
CREATE TABLE restaurants ( restaurant_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL, city VARCHAR(50), cuisine_type VARCHAR(50), rating DECIMAL(3,1) );  
[ Edit inline ] [ Edit ] [ Create PHP code ]  
  
✓ 6 rows inserted.  
Inserted row id: 6 (Query took 0.0069 seconds.)  
  
INSERT INTO restaurants (name, city, cuisine_type, rating) VALUES ('Spice Garden', 'Surat', 'Indian', 4.5), ('Royal Dine', 'Mumbai', 'Chinese', 4.2), ('Food Palace', 'Navsari', 'Gujarati', 4.7),  
( 'Urban Tadka', 'Surat', 'Punjabi', 4.1), ( 'Taste Hub', 'Ahmedabad', 'Italian', 4.6), ( 'Green Leaf', 'Mumbai', 'South Indian', 4.0);  
[ Edit inline ] [ Edit ] [ Create PHP code ]
```

✓ 1 row affected. (Query took 0.0010 seconds.)

`UPDATE restaurants SET rating = 4.8 WHERE restaurant_id = 1;`

[Edit inline] [Edit] [Create PHP code]

✓ 1 row deleted. (Query took 0.0007 seconds.)

`DELETE FROM restaurants WHERE restaurant_id = 6;`

[Edit inline] [Edit] [Create PHP code]

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

✓ Showing rows 0 - 4 (5 total, Query took 0.0005 seconds.) [rating: 4.8... - 4.1...]

`SELECT * FROM restaurants ORDER BY rating DESC;`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 ▾ | Filter rows:

Extra options

restaurant_id	name	city	cuisine_type	rating ▾ 1
1	Spice Garden	Surat	Indian	4.8
3	Food Palace	Navsari	Gujarati	4.7
5	Taste Hub	Ahmedabad	Italian	4.6
2	Royal Dine	Mumbai	Chinese	4.2
4	Urban Tadka	Surat	Punjabi	4.1

☐ Show all | Number of rows: 25 ▾ | Filter rows:

Task 2: Aggregate Revenue Reporting on Orders

ANS:

```
USE foodapp_db;
```

```
DROP TABLE IF EXISTS orders;
```

```
CREATE TABLE orders (  
    order_id INT AUTO_INCREMENT PRIMARY KEY,  
    restaurant_id INT,  
    customer_name VARCHAR(100),  
    total_amount DECIMAL(10,2),  
    order_date DATE  
);
```

```
INSERT INTO orders (restaurant_id, customer_name, total_amount,  
order_date)
```

```
VALUES
```

```
(1, 'Rahul', 1200.00, '2026-08-25'),  
(1, 'Amit', 1800.00, '2026-08-27'),  
(1, 'Priya', 2500.00, '2026-08-29'),  
(2, 'Neha', 2200.00, '2026-08-24'),  
(2, 'Rohan', 1500.00, '2026-08-28'),  
(2, 'Kiran', 1800.00, '2026-09-01'),  
(3, 'Mehul', 3000.00, '2026-08-26'),  
(3, 'Pooja', 2500.00, '2026-08-30'),
```

```
(4, 'Vivek', 1700.00, '2026-08-23'),  
(4, 'Anjali', 2100.00, '2026-08-31'),  
(4, 'Sahil', 1600.00, '2026-09-02'),  
(1, 'Riya', 1000.00, '2026-09-03');
```

-- Total revenue and average order value per restaurant

```
SELECT  
    restaurant_id,  
    SUM(total_amount) AS total_revenue,  
    AVG(total_amount) AS average_order_value  
FROM orders  
GROUP BY restaurant_id;
```

-- Restaurants whose total revenue exceeds Rs 5,000

```
SELECT  
    restaurant_id,  
    SUM(total_amount) AS total_revenue  
FROM orders  
GROUP BY restaurant_id  
HAVING SUM(total_amount) > 5000;
```

-- 5 most recent orders

```
SELECT *  
FROM orders  
ORDER BY order_date DESC
```

LIMIT 5;

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

USE foodapp_db;

[Edit inline][Edit][Create PHP code]

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0022 seconds.)

DROP TABLE IF EXISTS orders;

[Edit inline][Edit][Create PHP code]

⚠ Note: #1051 Unknown table 'foodapp_db.orders'

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0187 seconds.)

CREATE TABLE orders (order_id INT AUTO_INCREMENT PRIMARY KEY, restaurant_id INT, customer_name VARCHAR(100), total_amount DECIMAL(10,2), order_date DATE);

[Edit inline][Edit][Create PHP code]

✔ 12 rows inserted.
Inserted row id: 12 (Query took 0.0013 seconds.)

INSERT INTO orders (restaurant_id, customer_name, total_amount, order_date) VALUES (1, 'Rahul', 1200.00, '2026-08-25'), (1, 'Amit', 1800.00, '2026-08-27'), (1, 'Priya', 2500.00, '2026-08-29'), (2, 'Neha', 2200.00, '2026-08-24'), (2, 'Rohan', 1500.00, '2026-08-28'), (2, 'Kiran', 1800.00, '2026-09-01'), (3, 'Mehul', 3000.00, '2026-08-26'), (3, 'Pooja', 2500.00, '2026-08-30'), (4, 'Vivek', 1700.00, '2026-08-23'), (4, 'Anjali', 2100.00, '2026-08-31'), (4, 'Sahil', 1600.00, '2026-09-02'), (1, 'Riya', 1000.00, '2026-09-03');

[Edit inline][Edit][Create PHP code]

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

✔ Showing rows 0 - 3 (4 total, Query took 0.0023 seconds.)

-- Total revenue and average order value per restaurant SELECT restaurant_id, SUM(total_amount) AS total_revenue, AVG(total_amount) AS average_order_value FROM orders GROUP BY restaurant_id;

[Edit inline][Edit][Create PHP code]

☐ Show all

Number of rows: 25

Filter rows:

Extra options

restaurant_id	total_revenue	average_order_value
1	6500.00	1625.000000
2	5500.00	1833.333333
3	5500.00	2750.000000
4	5400.00	1800.000000

☐ Show all

Number of rows: 25

Filter rows:

✓ Showing rows 0 - 3 (4 total, Query took 0.0016 seconds.)

-- Restaurants whose total revenue exceeds Rs 5,000 `SELECT restaurant_id, SUM(total_amount) AS total_revenue FROM orders GROUP BY restaurant_id HAVING SUM(total_amount) > 5000;`

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

restaurant_id	total_revenue
1	6500.00
2	5500.00
3	5500.00
4	5400.00

☐ Show all | Number of rows: 25 ▾ Filter rows:

✓ Showing rows 0 - 4 (5 total, Query took 0.0014 seconds.) [order_date: 2026-09-03... - 2026-08-30...]

-- 5 most recent orders `SELECT * FROM orders ORDER BY order_date DESC LIMIT 5;`

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

Extra options

order_id	restaurant_id	customer_name	total_amount	order_date
12	1	Riya	1000.00	2026-09-03
11	4	Sahil	1600.00	2026-09-02
6	2	Kiran	1800.00	2026-09-01
10	4	Anjali	2100.00	2026-08-31
8	3	Pooja	2500.00	2026-08-30

Query results operations

 Print

 Copy to clipboard

 Export

 Display chart

 Create view

Task 3: Multi-Table JOINS and Menu Summary View

ANS:

```
USE foodapp_db;
```

```
CREATE TABLE restaurants (  
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    city VARCHAR(50)  
);
```

```
CREATE TABLE menu_items (  
    item_id INT AUTO_INCREMENT PRIMARY KEY,  
    restaurant_id INT,  
    item_name VARCHAR(100) NOT NULL,  
    price DECIMAL(10,2),  
    category VARCHAR(50),  
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)  
);
```

```
INSERT INTO restaurants (name, city)  
VALUES  
('Spice Garden', 'Surat'),  
('Royal Dine', 'Mumbai'),  
('Food Palace', 'Navsari'),  
('Urban Tadka', 'Ahmedabad'),
```

('Green Leaf', 'Vadodara');

INSERT INTO menu_items (restaurant_id, item_name, price, category)

VALUES

(1, 'Paneer Tikka', 250.00, 'Starter'),

(1, 'Butter Naan', 80.00, 'Bread'),

(1, 'Veg Biryani', 220.00, 'Main Course'),

(2, 'Hakka Noodles', 200.00, 'Main Course'),

(2, 'Manchurian', 180.00, 'Starter'),

(2, 'Fried Rice', 190.00, 'Main Course'),

(3, 'Gujarati Thali', 350.00, 'Thali'),

(3, 'Dhokla', 100.00, 'Snack'),

(3, 'Khandvi', 120.00, 'Snack'),

(4, 'Paneer Tikka Masala', 280.00, 'Main Course'),

(4, 'Tandoori Roti', 50.00, 'Bread'),

(4, 'Lassi', 100.00, 'Beverage');

-- INNER JOIN: Restaurants with menu items

SELECT

r.name AS restaurant_name,

m.item_name,

m.price

```
FROM restaurants r
INNER JOIN menu_items m
ON r.restaurant_id = m.restaurant_id;
```

-- LEFT JOIN: All restaurants including those without menu items

```
SELECT
    r.name AS restaurant_name,
    m.item_name,
    m.price
FROM restaurants r
LEFT JOIN menu_items m
ON r.restaurant_id = m.restaurant_id;
```

-- Create summary VIEW

```
CREATE VIEW restaurant_menu_summary AS
SELECT
    r.name AS restaurant_name,
    COUNT(m.item_id) AS total_menu_items,
    AVG(m.price) AS average_item_price
FROM restaurants r
LEFT JOIN menu_items m
ON r.restaurant_id = m.restaurant_id
GROUP BY r.restaurant_id, r.name;
```

-- Display VIEW

SELECT * FROM restaurant_menu_summary;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

USE foodapp_db;

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0024 seconds.)

DROP VIEW IF EXISTS restaurant_menu_summary;

[Edit inline] [Edit] [Create PHP code]

Note: #1051 Unknown table 'foodapp_db.restaurant_menu_summary'

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0009 seconds.)

DROP TABLE IF EXISTS menu_items;

[Edit inline] [Edit] [Create PHP code]

Note: #1051 Unknown table 'foodapp_db.menu_items'

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0114 seconds.)

DROP TABLE IF EXISTS restaurants;

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0039 seconds.)

CREATE TABLE restaurants (restaurant_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL, city VARCHAR(50));

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0045 seconds.)

CREATE TABLE menu_items (item_id INT AUTO_INCREMENT PRIMARY KEY, restaurant_id INT, item_name VARCHAR(100) NOT NULL, price DECIMAL(10,2), category VARCHAR(50), FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id));

[Edit inline] [Edit] [Create PHP code]

✓ 5 rows inserted.
Inserted row id: 5 (Query took 0.0010 seconds.)

INSERT INTO restaurants (name, city) VALUES ('Spice Garden', 'Surat'), ('Royal Dine', 'Mumbai'), ('Food Palace', 'Navsari'), ('Urban Tadka', 'Ahmedabad'), ('Green Leaf', 'Vadodara');

[Edit inline] [Edit] [Create PHP code]

✓ 12 rows inserted.
Inserted row id: 12 (Query took 0.0013 seconds.)

INSERT INTO menu_items (restaurant_id, item_name, price, category) VALUES (1, 'Paneer Tikka', 250.00, 'Starter'), (1, 'Butter Naan', 80.00, 'Bread'), (1, 'Veg Biryani', 220.00, 'Main Course'), (2, 'Makka Noodles', 200.00, 'Main Course'), (2, 'Manchurian', 180.00, 'Starter'), (2, 'Fried Rice', 190.00, 'Main Course'), (3, 'Gujarati Thali', 350.00, 'Thali'), (3, 'Dhokla', 100.00, 'Snack'), (3, 'Khandvi', 120.00, 'Snack'), (4, 'Paneer Tikka Masala', 280.00, 'Main Course'), (4, 'Tandoori Roti', 50.00, 'Bread'), (4, 'Lassi', 100.00, 'Beverage');

[Edit inline] [Edit] [Create PHP code]

Note: Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

✓ Showing rows 0 - 11 (12 total, Query took 0.0005 seconds.)

-- INNER JOIN: Restaurants with menu items SELECT r.name AS restaurant_name, m.item_name, m.price FROM restaurants r INNER JOIN menu_items m ON r.restaurant_id = m.restaurant_id;

[Edit inline] [Edit] [Create PHP code]

☐ Show all
 | Number of rows: 25
 Filter rows:

Extra options

restaurant_name	item_name	price
Spice Garden	Paneer Tikka	250.00
Spice Garden	Butter Naan	80.00
Spice Garden	Veg Biryani	220.00
Royal Dine	Hakka Noodles	200.00
Royal Dine	Manchurian	180.00
Royal Dine	Fried Rice	190.00
Food Palace	Gujarati Thali	350.00
Food Palace	Dhokla	100.00
Food Palace	Khandvi	120.00
Urban Tadka	Paneer Tikka Masala	280.00
Urban Tadka	Tandoori Roti	50.00
Urban Tadka	Lassi	100.00

☐ Show all
 | Number of rows: 25
 Filter rows:

Showing rows 0 - 12 (13 total, Query took 0.0005 seconds.)


```
-- LEFT JOIN: All restaurants including those without menu items
SELECT r.name AS restaurant_name, m.item_name, m.price FROM restaurants r
LEFT JOIN menu_items m ON r.restaurant_id = m.restaurant_id;
```

[\[Edit inline \]](#)
[\[Edit \]](#)
[\[Create PHP code \]](#)

☐ Show all
 | Number of rows: 25
 Filter rows:

restaurant_name	item_name	price
Spice Garden	Paneer Tikka	250.00
Spice Garden	Butter Naan	80.00
Spice Garden	Veg Biryani	220.00
Royal Dine	Hakka Noodles	200.00
Royal Dine	Manchurian	180.00
Royal Dine	Fried Rice	190.00
Food Palace	Gujarati Thali	350.00
Food Palace	Dhokla	100.00
Food Palace	Khandvi	120.00
Urban Tadka	Paneer Tikka Masala	280.00
Urban Tadka	Tandoori Roti	50.00
Urban Tadka	Lassi	100.00
Green Leaf	NULL	NULL

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0039 seconds.)

-- Create summary VIEW CREATE VIEW restaurant_menu_summary AS SELECT r.name AS restaurant_name, COUNT(m.item_id) AS total_menu_items, AVG(m.price) AS average_item_price FROM restaurants r LEFT JOIN menu_items m ON r.restaurant_id = m.restaurant_id GROUP BY r.restaurant_id, r.name;

[Edit inline] [Edit] [Create PHP code]

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

✔ Showing rows 0 - 4 (5 total, Query took 0.0010 seconds.)

-- Display VIEW SELECT * FROM restaurant_menu_summary;

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 ▼ Filter rows:

Extra options

restaurant_name	total_menu_items	average_item_price
Spice Garden	3	183.333333
Royal Dine	3	190.000000
Food Palace	3	190.000000
Urban Tadka	3	143.333333
Green Leaf	0	NULL

☐ Show all | Number of rows: 25 ▼ Filter rows:

Task 4: PL/SQL Order Placement Procedure

ANS:

```
USE foodapp_db;
```

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    wallet_balance DECIMAL(10,2)  
);
```

```
CREATE TABLE orders (  
    order_id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_id INT,  
    restaurant_id INT,  
    amount DECIMAL(10,2),  
    order_date DATE,  
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

```
INSERT INTO customers (customer_id, customer_name, wallet_balance)  
VALUES  
(1, 'Rahul', 10000.00),  
(2, 'Amit', 3000.00);
```

DELIMITER //

CREATE PROCEDURE place_order(

IN p_customer_id INT,

IN p_restaurant_id INT,

IN p_amount DECIMAL(10,2)

)

BEGIN

DECLARE current_balance DECIMAL(10,2);

START TRANSACTION;

SELECT wallet_balance

INTO current_balance

FROM customers

WHERE customer_id = p_customer_id

FOR UPDATE;

IF current_balance >= p_amount THEN

UPDATE customers

SET wallet_balance = wallet_balance - p_amount

WHERE customer_id = p_customer_id;

```
INSERT INTO orders (customer_id, restaurant_id, amount,
order_date)
VALUES (p_customer_id, p_restaurant_id, p_amount, CURDATE());

COMMIT;

SELECT 'Order placed successfully' AS message,
       p_amount AS amount,
       current_balance - p_amount AS remaining_balance;

ELSE

ROLLBACK;

SELECT 'Insufficient wallet balance - Order cancelled' AS message,
       current_balance AS wallet_balance,
       p_amount AS requested_amount;

END IF;
END //

DELIMITER ;

-- TEST 1: Valid order
CALL place_order(1, 1, 2500.00);
```

-- TEST 2: Order exceeding wallet balance

CALL place_order(2, 1, 5000.00);

-- Verify customers

SELECT * FROM customers;

-- Verify orders

SELECT * FROM orders;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

USE foodapp_db;

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0251 seconds.)

DROP PROCEDURE IF EXISTS place_order;

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

⚠ Note: #1305 PROCEDURE foodapp_db.place_order does not exist

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0075 seconds.)

DROP TABLE IF EXISTS orders;

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)

DROP TABLE IF EXISTS customers;

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

⚠ Note: #1051 Unknown table 'foodapp_db.customers'

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0041 seconds.)

CREATE TABLE customers (customer_id INT PRIMARY KEY, customer_name VARCHAR(100), wallet_balance DECIMAL(10,2));

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0033 seconds.)

```
CREATE TABLE orders ( order_id INT AUTO_INCREMENT PRIMARY KEY, customer_id INT, restaurant_id INT, amount DECIMAL(10,2), order_date DATE, FOREIGN KEY (customer_id) REFERENCES customers(customer_id) );
```

[Edit inline] [Edit] [Create PHP code]

✓ 2 rows inserted. (Query took 0.0009 seconds.)

```
INSERT INTO customers (customer_id, customer_name, wallet_balance) VALUES (1, 'Rahul', 10000.00), (2, 'Amit', 3000.00);
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0066 seconds.)

```
CREATE PROCEDURE place_order( IN p_customer_id INT, IN p_restaurant_id INT, IN p_amount DECIMAL(10,2) ) BEGIN DECLARE current_balance DECIMAL(10,2); START TRANSACTION; SELECT wallet_balance INTO current_balance FROM customers WHERE customer_id = p_customer_id FOR UPDATE; IF current_balance <= p_amount THEN UPDATE customers SET wallet_balance = wallet_balance - p_amount WHERE customer_id = p_customer_id; INSERT INTO orders (customer_id, restaurant_id, amount, order_date) VALUES (p_customer_id, p_restaurant_id, p_amount, CURDATE()); COMMIT; SELECT 'Order placed successfully' AS message, p_amount AS amount, current_balance - p_amount AS remaining_balance; ELSE ROLLBACK; SELECT 'Insufficient wallet balance - Order cancelled' AS message, current_balance AS wallet_balance, [...]
```

[Edit]

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

✓ Showing rows 0 - 0 (1 total, Query took 0.0054 seconds.)

```
-- TEST 1: Valid order CALL place_order(1, 1, 2500.00);
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 ▾ | Filter rows:

Extra options

message	amount	remaining_balance
Order placed successfully	2500.00	7500.00

☐ Show all | Number of rows: 25 ▾ | Filter rows:

✓ Showing rows 0 - 0 (1 total, Query took 0.0005 seconds.)

```
-- TEST 2: Order exceeding wallet balance CALL place_order(2, 1, 5000.00);
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 ▾ | Filter rows:

Extra options

message	wallet_balance	requested_amount
Insufficient wallet balance - Order cancelled	3000.00	5000.00

☐ Show all | Number of rows: 25 ▾ | Filter rows:

Query results operations

✓ Showing rows 0 - 1 (2 total, Query took 0.0004 seconds.)

-- Verify customers SELECT * FROM customers;

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

customer_id	customer_name	wallet_balance
1	Rahul	7500.00
2	Amit	3000.00

☐ Show all | Number of rows: 25 ▾ Filter rows:

✓ Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)

-- Verify orders SELECT * FROM orders;

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

order_id	customer_id	restaurant_id	amount	order_date
1	1	1	2500.00	2026-09-06

☐ Show all | Number of rows: 25 ▾ Filter rows:

Query results operations

Section C — Mini Capstone Project

Mini Project: Food Delivery Platform — Backend Database

ANS:

```
CREATE DATABASE IF NOT EXISTS foodapp_db;
```

```
USE foodapp_db;
```

```
-- RESTAURANTS TABLE
```

```
CREATE TABLE restaurants (  
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    cuisine_type VARCHAR(50) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- MENU ITEMS TABLE
```

```
CREATE TABLE menu_items (  
    item_id INT AUTO_INCREMENT PRIMARY KEY,  
    restaurant_id INT NOT NULL,  
    item_name VARCHAR(100) NOT NULL,  
    price DECIMAL(10,2) NOT NULL,  
    category VARCHAR(50) NOT NULL,  
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)  
);
```

-- ORDERS TABLE

```
CREATE TABLE orders (  
    order_id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_name VARCHAR(100) NOT NULL,  
    restaurant_id INT NOT NULL,  
    item_id INT NOT NULL,  
    quantity INT NOT NULL,  
    total_amount DECIMAL(10,2) NOT NULL,  
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id),  
    FOREIGN KEY (item_id) REFERENCES menu_items(item_id)  
);
```

-- ORDER AUDIT TABLE

```
CREATE TABLE order_audit (  
    audit_id INT AUTO_INCREMENT PRIMARY KEY,  
    order_id INT NOT NULL,  
    restaurant_id INT NOT NULL,  
    action VARCHAR(20) NOT NULL,  
    log_time DATETIME NOT NULL  
);
```

-- INSERT RESTAURANTS

```
INSERT INTO restaurants (name, city, cuisine_type)
```

VALUES

('Spice Garden', 'Surat', 'Indian'),
('Royal Dine', 'Mumbai', 'Chinese'),
('Food Palace', 'Navsari', 'Gujarati');

-- INSERT MENU ITEMS

INSERT INTO menu_items (restaurant_id, item_name, price, category)

VALUES

(1, 'Paneer Tikka', 250.00, 'Starter'),
(1, 'Veg Biryani', 220.00, 'Main Course'),
(1, 'Butter Naan', 80.00, 'Bread'),
(1, 'Mango Lassi', 120.00, 'Beverage'),

(2, 'Hakka Noodles', 200.00, 'Main Course'),
(2, 'Manchurian', 180.00, 'Starter'),
(2, 'Fried Rice', 190.00, 'Main Course'),
(2, 'Spring Roll', 150.00, 'Starter'),

(3, 'Gujarati Thali', 350.00, 'Main Course'),
(3, 'Dhokla', 100.00, 'Starter'),
(3, 'Khandvi', 120.00, 'Starter'),
(3, 'Chaas', 60.00, 'Beverage');

-- TRIGGER

DELIMITER //

```
CREATE TRIGGER after_order_insert
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
    INSERT INTO order_audit
        (order_id, restaurant_id, action, log_time)
    VALUES
        (NEW.order_id, NEW.restaurant_id, 'INSERT', NOW());
END //
```

```
DELIMITER ;
```

```
-- 15 ORDER RECORDS
```

```
INSERT INTO orders
(customer_name, restaurant_id, item_id, quantity, total_amount)
VALUES
('Rahul', 1, 1, 2, 500.00),
('Amit', 1, 2, 1, 220.00),
('Priya', 1, 3, 3, 240.00),
('Neha', 1, 4, 2, 240.00),
('Rohan', 1, 1, 1, 250.00),

('Kiran', 2, 5, 2, 400.00),
('Pooja', 2, 6, 1, 180.00),
```

('Vivek', 2, 7, 2, 380.00),

('Anjali', 2, 8, 3, 450.00),

('Sahil', 2, 5, 1, 200.00),

('Mehul', 3, 9, 1, 350.00),

('Riya', 3, 10, 3, 300.00),

('Dhruv', 3, 11, 2, 240.00),

('Nisha', 3, 12, 2, 120.00),

('Harsh', 3, 9, 2, 700.00);

-- VIEW

CREATE VIEW restaurant_sales_summary AS

SELECT

 r.name AS restaurant_name,

 COUNT(o.order_id) AS total_orders,

 COALESCE(SUM(o.total_amount), 0) AS total_revenue

FROM restaurants r

LEFT JOIN orders o

 ON r.restaurant_id = o.restaurant_id

GROUP BY r.restaurant_id, r.name;

-- STORED PROCEDURE

DELIMITER //

CREATE PROCEDURE add_order(

```
IN p_customer_name VARCHAR(100),
IN p_restaurant_id INT,
IN p_item_id INT,
IN p_quantity INT
)
BEGIN
    DECLARE restaurant_count INT DEFAULT 0;
    DECLARE item_price DECIMAL(10,2);
    DECLARE item_restaurant INT;
    DECLARE total DECIMAL(10,2);

    START TRANSACTION;

    SELECT COUNT(*)
    INTO restaurant_count
    FROM restaurants
    WHERE restaurant_id = p_restaurant_id;

    IF restaurant_count = 0 THEN
        ROLLBACK;
        SELECT 'Restaurant does not exist. Order cancelled.' AS message;
    ELSE

        SELECT price, restaurant_id
```

```
INTO item_price, item_restaurant
FROM menu_items
WHERE item_id = p_item_id;

IF item_restaurant <> p_restaurant_id THEN
    ROLLBACK;

    SELECT 'Menu item does not belong to this restaurant.' AS
message;

ELSE

    SET total = item_price * p_quantity;

    INSERT INTO orders
(customer_name, restaurant_id, item_id, quantity, total_amount)
VALUES
(p_customer_name, p_restaurant_id, p_item_id, p_quantity,
total);

    COMMIT;

    SELECT
        'Order added successfully' AS message,
        total AS total_amount;

END IF;

END IF;
```

END //

DELIMITER ;

-- TEST PROCEDURE

CALL add_order('Test Customer', 1, 1, 2);

-- TEST INVALID RESTAURANT

CALL add_order('Invalid Customer', 99, 1, 2);

-- DISPLAY SALES SUMMARY

SELECT * FROM restaurant_sales_summary;

-- DISPLAY ORDERS

SELECT * FROM orders;

-- DISPLAY AUDIT LOG

SELECT * FROM order_audit;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0008 seconds.)

```
CREATE DATABASE IF NOT EXISTS foodapp_db;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

⚠ Note: #1007 Can't create database 'foodapp_db'; database exists

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
USE foodapp_db;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0007 seconds.)

```
DROP TRIGGER IF EXISTS after_order_insert;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

⚠ Note: #1360 Trigger does not exist

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)

```
DROP PROCEDURE IF EXISTS add_order;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

⚠ Note: #1305 PROCEDURE foodapp_db.add_order does not exist

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)

```
DROP VIEW IF EXISTS restaurant_sales_summary;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0009 seconds.)

```
DROP TABLE IF EXISTS order_audit;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

⚠ Note: #1051 Unknown table 'foodapp_db.order_audit'

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0299 seconds.)

```
DROP TABLE IF EXISTS orders;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0052 seconds.)

```
DROP TABLE IF EXISTS menu_items;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0046 seconds.)

```
DROP TABLE IF EXISTS restaurants;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0068 seconds.)

```
-- RESTAURANTS TABLE CREATE TABLE restaurants ( restaurant_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL, city VARCHAR(50) NOT NULL, cuisine_type VARCHAR(50) NOT NULL, created_at  
TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0062 seconds.)

```
-- MENU ITEMS TABLE CREATE TABLE menu_items ( item_id INT AUTO_INCREMENT PRIMARY KEY, restaurant_id INT NOT NULL, item_name VARCHAR(100) NOT NULL, price DECIMAL(10,2) NOT NULL, category  
VARCHAR(50) NOT NULL, FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id) );
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0052 seconds.)

```
-- ORDERS TABLE CREATE TABLE orders ( order_id INT AUTO_INCREMENT PRIMARY KEY, customer_name VARCHAR(100) NOT NULL, restaurant_id INT NOT NULL, item_id INT NOT NULL, quantity INT NOT NULL,  
total_amount DECIMAL(10,2) NOT NULL, order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id), FOREIGN KEY (item_id) REFERENCES  
menu_items(item_id) );
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0045 seconds.)

```
-- ORDER AUDIT TABLE CREATE TABLE order_audit ( audit_id INT AUTO_INCREMENT PRIMARY KEY, order_id INT NOT NULL, restaurant_id INT NOT NULL, action VARCHAR(20) NOT NULL, log_time DATETIME NOT  
NULL );
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✔ 3 rows inserted.

Inserted row id: 3 (Query took 0.0008 seconds.)

```
-- INSERT RESTAURANTS INSERT INTO restaurants (name, city, cuisine_type) VALUES ('Spice Garden', 'Surat', 'Indian'), ('Royal Dine', 'Mumbai', 'Chinese'), ('Food Palace', 'Navsari', 'Gujarati');
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✔ 12 rows inserted.
Inserted row id: 12 (Query took 0.0012 seconds.)

```
-- INSERT MENU ITEMS INSERT INTO menu_items (restaurant_id, item_name, price, category) VALUES (1, 'Paneer Tikka', 250.00, 'Starter'), (1, 'Veg Biryani', 220.00, 'Main Course'), (1, 'Butter Naan', 80.00, 'Bread'), (1, 'Mango Lassi', 120.00, 'Beverage'), (2, 'Hakka Noodles', 200.00, 'Main Course'), (2, 'Manchurian', 180.00, 'Starter'), (2, 'Fried Rice', 190.00, 'Main Course'), (2, 'Spring Roll', 150.00, 'Starter'), (3, 'Gujarati Thali', 350.00, 'Main Course'), (3, 'Dhokla', 100.00, 'Starter'), (3, 'Khandvi', 120.00, 'Starter'), (3, 'Chaos', 60.00, 'Beverage');
```

[Edit inline] [Edit] [Create PHP code]

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0045 seconds.)

```
CREATE TRIGGER after_order_insert AFTER INSERT ON orders FOR EACH ROW BEGIN INSERT INTO order_audit (order_id, restaurant_id, action, log_time) VALUES (NEW.order_id, NEW.restaurant_id, 'INSERT', NOW()); END;
```

[Edit inline] [Edit] [Create PHP code]

✔ 15 rows inserted.
Inserted row id: 15 (Query took 0.0022 seconds.)

```
-- 15 ORDER RECORDS INSERT INTO orders (customer_name, restaurant_id, item_id, quantity, total_amount) VALUES ('Rahul', 1, 1, 2, 500.00), ('Amit', 1, 2, 1, 220.00), ('Priya', 1, 3, 3, 240.00), ('Neha', 1, 4, 2, 240.00), ('Rohan', 1, 1, 1, 250.00), ('Kiran', 2, 5, 2, 400.00), ('Pooja', 2, 6, 1, 180.00), ('Vivek', 2, 7, 2, 380.00), ('Anjali', 2, 8, 3, 450.00), ('Sahil', 2, 5, 1, 200.00), ('Mehul', 3, 9, 1, 350.00), ('Riya', 3, 10, 3, 300.00), ('Dhruv', 3, 11, 2, 240.00), ('Nisha', 3, 12, 2, 120.00), ('Harsh', 3, 9, 2, 700.00);
```

[Edit inline] [Edit] [Create PHP code]

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0023 seconds.)

```
-- VIEW CREATE VIEW restaurant_sales_summary AS SELECT r.name AS restaurant_name, COUNT(o.order_id) AS total_orders, COALESCE(SUM(o.total_amount), 0) AS total_revenue FROM restaurants r LEFT JOIN orders o ON r.restaurant_id = o.restaurant_id GROUP BY r.restaurant_id, r.name;
```

[Edit inline] [Edit] [Create PHP code]

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0023 seconds.)

```
-- VIEW CREATE VIEW restaurant_sales_summary AS SELECT r.name AS restaurant_name, COUNT(o.order_id) AS total_orders, COALESCE(SUM(o.total_amount), 0) AS total_revenue FROM restaurants r LEFT JOIN orders o ON r.restaurant_id = o.restaurant_id GROUP BY r.restaurant_id, r.name;
```

[Edit inline] [Edit] [Create PHP code]

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0029 seconds.)

```
CREATE PROCEDURE add_order( IN p_customer_name VARCHAR(100), IN p_restaurant_id INT, IN p_item_id INT, IN p_quantity INT ) BEGIN DECLARE restaurant_count INT DEFAULT 0; DECLARE item_price DECIMAL(10,2); DECLARE item_restaurant INT; DECLARE total DECIMAL(10,2); START TRANSACTION; SELECT COUNT(*) INTO restaurant_count FROM restaurants WHERE restaurant_id = p_restaurant_id; IF restaurant_count = 0 THEN ROLLBACK; SELECT 'Restaurant does not exist. Order cancelled.' AS message; ELSE SELECT price, restaurant_id INTO item_price, item_restaurant FROM menu_items WHERE item_id = p_item_id; IF item_restaurant <> p_restaurant_id THEN ROLLBACK; SELECT 'Menu item does not belong to this restaurant.' AS message; ELSE SET total = item_price * p_quantity; INSERT INTO orders [...]
```

[Edit]

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

✔ Showing rows 0 - 0 (1 total, Query took 0.0010 seconds.)

```
-- TEST PROCEDURE CALL add_order('Test Customer', 1, 1, 2);
```

[Edit inline] [Edit] [Create PHP code]

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

message

Restaurant does not exist. Order cancelled.

☐ Show all | Number of rows: 25 ▾ Filter rows:

✓ Showing rows 0 - 2 (3 total, Query took 0.0010 seconds.)

-- DISPLAY SALES SUMMARY SELECT * FROM restaurant_sales_summary;

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

restaurant_name	total_orders	total_revenue
Spice Garden	6	1950.00
Royal Dine	5	1610.00
Food Palace	5	1710.00

☐ Show all | Number of rows: 25 ▾ Filter rows:

✓ Showing rows 0 - 15 (16 total, Query took 0.0004 seconds.)

-- DISPLAY ORDERS SELECT * FROM orders;

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

order_id	customer_name	restaurant_id	item_id	quantity	total_amount	order_date
1	Rahul	1	1	2	500.00	2026-09-06 11:38:42
2	Amit	1	2	1	220.00	2026-09-06 11:38:42
3	Priya	1	3	3	240.00	2026-09-06 11:38:42
4	Neha	1	4	2	240.00	2026-09-06 11:38:42
5	Rohan	1	1	1	250.00	2026-09-06 11:38:42
6	Kiran	2	5	2	400.00	2026-09-06 11:38:42
7	Pooja	2	6	1	180.00	2026-09-06 11:38:42
8	Vivek	2	7	2	380.00	2026-09-06 11:38:42
9	Anjali	2	8	3	450.00	2026-09-06 11:38:42
10	Sahil	2	5	1	200.00	2026-09-06 11:38:42
11	Mehul	3	9	1	350.00	2026-09-06 11:38:42
12	Riya	3	10	3	300.00	2026-09-06 11:38:42
13	Dhruv	3	11	2	240.00	2026-09-06 11:38:42
14	Nisha	3	12	2	120.00	2026-09-06 11:38:42
15	Harsh	3	9	2	700.00	2026-09-06 11:38:42
16	Test Customer	1	1	2	500.00	2026-09-06 11:38:42

☐ Show all | Number of rows: 25 ▾ Filter rows:

✓ Showing rows 0 - 15 (16 total, Query took 0.0007 seconds.)

-- DISPLAY AUDIT LOG SELECT * FROM order_audit;

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: 25 ▼ Filter rows:

Extra options

audit_id	order_id	restaurant_id	action	log_time
1	1	1	1 INSERT	2026-09-06 11:38:42
2	2	2	1 INSERT	2026-09-06 11:38:42
3	3	3	1 INSERT	2026-09-06 11:38:42
4	4	4	1 INSERT	2026-09-06 11:38:42
5	5	5	1 INSERT	2026-09-06 11:38:42
6	6	6	2 INSERT	2026-09-06 11:38:42
7	7	7	2 INSERT	2026-09-06 11:38:42
8	8	8	2 INSERT	2026-09-06 11:38:42
9	9	9	2 INSERT	2026-09-06 11:38:42
10	10	10	2 INSERT	2026-09-06 11:38:42
11	11	11	3 INSERT	2026-09-06 11:38:42
12	12	12	3 INSERT	2026-09-06 11:38:42
13	13	13	3 INSERT	2026-09-06 11:38:42
14	14	14	3 INSERT	2026-09-06 11:38:42
15	15	15	3 INSERT	2026-09-06 11:38:42
16	16	16	1 INSERT	2026-09-06 11:38:42

☐ Show all | Number of rows: 25 ▼ Filter rows:

Section D — AI-Augmented Learning

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a stored procedure that

ANS:

Exact Prompt Given to AI :

Write a MySQL stored procedure named `restaurant_monthly_summary` that accepts a `restaurant_id` and a month number (1–12) as input parameters. It should return the total number of orders, total revenue, and average order value for that restaurant in the specified month. Use `SELECT` with `GROUP BY` and aggregate functions. If the restaurant has no orders in that month, return a meaningful message instead of an empty result. The procedure must run successfully in MySQL Workbench.

AI's Original Procedure:

```
USE foodapp_db;
```

```
DELIMITER //
```

```
CREATE PROCEDURE restaurant_monthly_summary(
```

```
    IN p_restaurant_id INT,
```

```
    IN p_month INT
```

```
)
```

```
BEGIN
```

```
    SELECT
```

```
    restaurant_id,  
    COUNT(order_id) AS total_orders,  
    SUM(total_amount) AS total_revenue,  
    AVG(total_amount) AS average_order_value  
FROM orders  
WHERE restaurant_id = p_restaurant_id  
    AND MONTH(order_date) = p_month  
GROUP BY restaurant_id;  
  
IF ROW_COUNT() = 0 THEN  
    SELECT 'No orders found for this restaurant in the selected month.'  
    AS message;  
END IF;  
END //
```

```
DELIMITER ;
```

Corrected :

```
USE foodapp_db;  
  
DROP PROCEDURE IF EXISTS restaurant_monthly_summary;  
  
DELIMITER //  
  
CREATE PROCEDURE restaurant_monthly_summary(  
    restaurant_id,
```

```
    IN p_restaurant_id INT,
    IN p_month INT
)
BEGIN
    DECLARE order_count INT DEFAULT 0;

    IF p_month < 1 OR p_month > 12 THEN

        SELECT 'Invalid month. Please enter a month between 1 and 12.'
        AS message;

    ELSE

        SELECT COUNT(*)
        INTO order_count
        FROM orders
        WHERE restaurant_id = p_restaurant_id
        AND MONTH(order_date) = p_month;

        IF order_count = 0 THEN

            SELECT
                'No orders found for this restaurant in the selected month.'
            AS message;
```

ELSE

SELECT

restaurant_id,

COUNT(order_id) AS total_orders,

SUM(total_amount) AS total_revenue,

AVG(total_amount) AS average_order_value

FROM orders

WHERE restaurant_id = p_restaurant_id

AND MONTH(order_date) = p_month

GROUP BY restaurant_id;

END IF;

END IF;

END //

DELIMITER ;

-- Test 1

CALL restaurant_monthly_summary(1, 8);

-- Test 2

CALL restaurant_monthly_summary(2, 8);

-- Test 3: No orders / meaningful message

CALL restaurant_monthly_summary(3, 12);

3-4 Line Note:

The AI version used ROW_COUNT() after the SELECT statement, which does not reliably indicate whether the SELECT returned any rows. I fixed this by using COUNT(*) INTO order_count before generating the summary. I also added validation for the month input so only values from 1 to 12 are accepted. This makes the procedure safer and ensures a meaningful message is returned when no orders exist.